

Reporte del Proyecto SIA

Gestión de Asistencia

Asignatura	INF2236 - Programación Avanzada
Profesor	Claudio Cubillos
Periodo	2do Semestre 2026
Integrantes	Nicolás Jaramillo, Lucas Ahumada, Alessandro Reginato
Fecha de entrega	11 de Septiembre de 2026
Versión del proyecto	Version Final

Resumen del proyecto

Problema que aborda el sistema: Gestión de Asistencia de un año escolar para un colegio

Usuarios del sistema: Funcionarios Escolares

Funcionalidades principales: Organizar y digitalizar asistencia para un colegio

Tecnologías utilizadas: JAVA 11, Github, Netbeans 21, CSV

Forma de ejecución: Consola y Ventana

Tabla de Contenidos

Sección	Página
Tabla de contenidos	1
SIA-1. Análisis de datos y funcionalidades	2
SIA-2. Diseño conceptual de clases	3
SIA-3. Buenas prácticas	4
SIA-4. Colecciones anidadas	5
SIA-5. Sobrecarga de métodos	6
SIA-6. Sobreescritura de métodos	7
SIA-7. Menú de inserción y listado	8
SIA-8. Menú de edición, eliminación y búsqueda	9
SIA-9. Funcionalidad de utilidad para el negocio	10
SIA-10. Consola y ventanas	11
SIA-11. Persistencia de datos	12
SIA-12. Excepciones propias y try-catch	13
SIA-13. Uso de GitHub	14

SIA-1 Análisis de datos y funcionalidades

Requisito solicitado: Realizar un análisis de los datos a utilizar y de las principales funcionalidades que dan sentido al proyecto.

Análisis de los datos

Nuestro sistema guarda información de los alumnos, los cursos y las asistencias. Con estos datos se puede organizar a los estudiantes y revisar su asistencia durante el año escolar.

Datos del alumno

Cada alumno tiene RUT, nombre y apellido, que se guardan como texto. El RUT puede llevar puntos, guión y un dígito verificador con letra, por eso no se guarda como número. Un alumno puede estar en un curso o quedar sin curso asignado.

Datos del curso

Cada curso tiene un código y un nombre. También tiene un profesor jefe, representado por un objeto de tipo Profesor, que guarda sus datos y la asignatura que enseña. El código permite encontrar el curso y distinguirlo de otros. Cada curso mantiene una lista de sus alumnos.

Datos de asistencia

Cada registro relaciona a un alumno con una fecha. La fecha representa el año, mes y día. También se guarda si el alumno estuvo presente o si tuvo una falta justificada o si se retiró antes. La justificación y el motivo del retiro se guardan como texto. Estos datos se mantienen separados porque un alumno puede faltar con justificación o estar presente y retirarse antes.

Historial y porcentaje de asistencia

Los registros de cada alumno se organizan usando su RUT, así se puede consultar su historial sin mezclarlo con el de otros estudiantes. Con estos registros, el sistema calcula el porcentaje de asistencia.

Funcionalidades principales

El sistema permite agregar alumnos y organizarlos en cursos, registrar la asistencia y consultar quiénes faltaron en una fecha. También permite revisar la asistencia de un alumno o curso y calcular el porcentaje de asistencia. Estas opciones ayudan a encontrar la información más rápido y a hacer seguimiento de las faltas.

Evidencia

Archivo: [README.md](#), sección "La idea": explica el objetivo general del sistema, la cual fue pensada al momento de crear el repositorio.

SIA-2 Diseño conceptual de clases

Requisito solicitado: Diseñar el modelo conceptual de clases del dominio mediante UML y codificarlo en Java.

Clases del dominio

1. Persona: contiene los datos comunes de las personas, como RUT, nombre y apellido.
2. Alumno: hereda los datos de Persona y puede estar asociado a un curso.
3. Profesor: hereda los datos de Persona y contiene la asignatura que enseña.
4. Curso: contiene su código, nombre, profesor jefe y alumnos.
5. Asistencia: registra la asistencia de un alumno en una fecha, incluyendo si estuvo presente, si la falta fue justificada o si se retiró antes, y los motivos correspondientes.

Relaciones

Persona es la clase base de Alumno y Profesor, por lo que ambas clases comparten sus datos personales. Cada curso tiene un profesor jefe y mantiene una lista de alumnos. Alumno guarda la referencia a su curso, y cada registro de asistencia corresponde a un alumno. Un alumno puede tener varios registros de asistencia.

Evidencia UML

El diagrama muestra las clases del sistema y sus relaciones, incluyendo el dominio, los gestores, los menús, las ventanas y las excepciones.

[Diagrama UML Clases SIA Asistencia REVISION.pdf](#)

SIA-3 Buenas prácticas

Requisito solicitado: Mantener atributos privados con getters y setters, código legible y modularizado, documentación interna útil y datos iniciales suficientes.

Encapsulamiento

Los atributos de nuestras clases son privados y se consultan o modifican mediante métodos públicos. Para proteger las colecciones, los métodos de consulta entregan copias de las listas y los mapas, evitando que otra parte del programa agregue o elimine elementos directamente en las colecciones internas.

En el registro de asistencia se copia tanto el mapa como las listas que contiene. Estas copias mantienen las referencias a los mismos alumnos y registros; protegen la estructura de las colecciones, pero no duplican los objetos.

La eliminación de un alumno de un curso se realiza mediante una operación del propio curso. El gestor de alumnos utiliza esa operación al cambiar al estudiante de curso o eliminarlo del sistema.

Organización del código

Organizamos el proyecto según las responsabilidades de sus clases. Las clases del dominio representan los datos del colegio, los gestores realizan las operaciones sobre ellos y los menús y ventanas reciben las solicitudes del usuario y presentan los resultados.

Los resúmenes de personas, alumnos y profesores, junto con los listados y detalles de alumnos y cursos, devuelven información en forma de texto sin imprimirla directamente. La consola y las ventanas utilizan esos mismos resultados y se encargan de mostrarlos. Esto permite compartir su elaboración entre ambas interfaces y evita mantener versiones separadas de los mismos listados.

Datos iniciales

El sistema incorpora datos iniciales mediante una clase dedicada a su carga. Se incluye un profesor jefe con su asignatura, un curso, dos alumnos y registros que representan presencia, ausencia, falta justificada y retiro anticipado. Estos datos permiten ejecutar las principales consultas de asistencia sin ingresar toda la información manualmente. Antes de agregar los registros de asistencia de ejemplo, se comprueba que no exista uno para el mismo alumno y fecha.

Evidencia principal: [src/main/java](#)

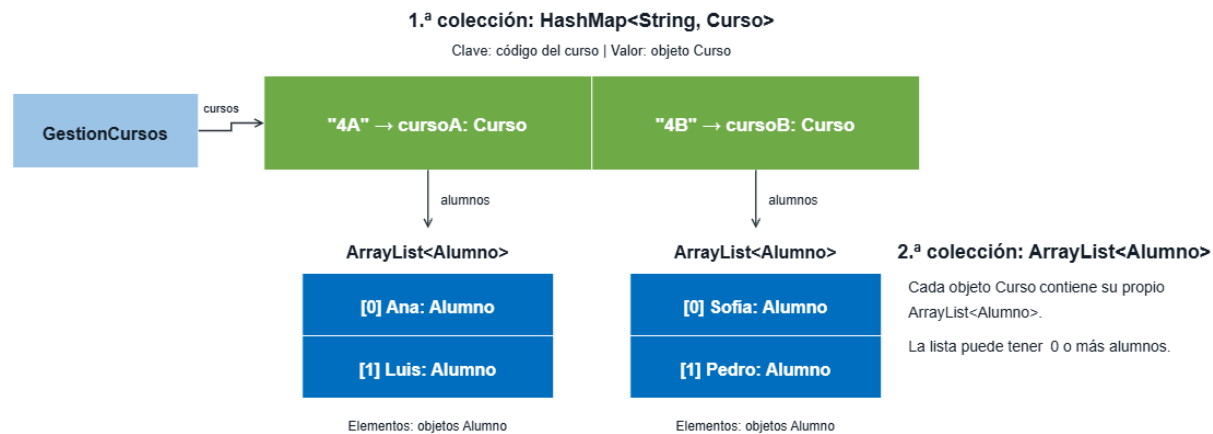
SIA-4 Colecciones anidadas

Requisito solicitado: Diseñar y codificar dos colecciones de objetos, con la segunda anidada; al menos una debe ser un mapa del Java Collections Framework.

Colecciones utilizadas y Anidamiento

En el proyecto usamos un mapa para guardar los cursos y una lista de alumnos dentro de cada curso. El mapa relaciona el código de cada curso con la información del curso, así se puede buscar directamente. La lista permite guardar una cantidad variable de alumnos y recorrerlos cuando se consulta un curso.

Representación de la anidación



En el ejemplo mostrado en el diagrama, se muestra el `HashMap` con dos elementos, un curso de clave “4A” y otro “4B”, el valor de cada elemento corresponde a un objeto de tipo `Curso`, objetos los cuales poseen `ArrayList` que contienen objetos tipo `Alumno` correspondiente al curso.

Evidencia

[src/main/java/GestionCursos.java](#), aproximadamente líneas 8 y 18.

[src/main/java/Curso.java](#), aproximadamente líneas 9 y 15.

[src/main/java/GestionRegistroAsistencia.java](#), aproximadamente línea 10.

Evidencia principal: [GestionCursos.java](#), [Curso.java](#) y [GestionRegistroAsistencia.java](#).

SIA-5 Sobrecarga de métodos

Requisito solicitado: Diseñar y codificar dos clases que utilicen sobrecarga de métodos, sin considerar sobrecarga de constructores. Ambas clases deben ser utilizadas.

Clases y métodos

1. **GestionAlumnos:** contiene dos versiones del método **buscarAlumno**, una que recibe un RUT y otra que recibe un nombre y apellido. Ambos métodos buscan un Alumno en la lista general de alumnos y lo retornar o null si no existe.
2. **Alumno:** contiene dos versiones de obtenerResumen, una que muestra los datos del alumno y otra que permite elegir un formato corto. Ambas entregan la información como texto, no la imprimen directamente.

Uso

buscarAlumno: es llamado desde el método **obtenerAlumno** que utiliza el menú cuando el usuario busca un alumno por RUT, también se utiliza como método de verificación en el agregado de alumnos, la versión que recibe nombre y apellido es llamada por el método **mostrarMenu** en la clase **MenuAlumnos** y desde **VentanaAlumnos**.

obtenerResumen: es llamado desde **MenuAlumnos** para mostrar el resultado de una búsqueda, la versión de formato corto se utiliza desde el método **obtenerListadoAlumnos** en la clase **GestionAlumnos**.

Evidencia

En [GestionAlumnos.java](#), buscarAlumno tiene una versión que busca por RUT y otra que busca por nombre y apellido. Estas búsquedas se utilizan desde [MenuAlumnos.java](#) y [VentanaAlumnos.java](#).

En [Alumno.java](#), obtenerResumen tiene una versión completa y otra que permite mostrar un formato corto. La versión completa se usa al consultar un alumno y la versión corta se utiliza desde GestionAlumnos.java al mostrar el listado.

SIA-6 Sobreescritura de métodos

Requisito solicitado: Diseñar y codificar dos clases que utilicen sobreescritura de métodos. Ambas clases deben ser utilizadas.

Clases

La clase Persona define el método public String obtenerResumen(), el cual retorna un resumen con el RUT, nombre y apellido de la persona.

Las clases Alumno y Profesor heredan de Persona y sobrescriben este método, manteniendo la misma firma pero entregando información específica según el tipo de objeto, en caso de un Alumno, incluye datos personales y el código del curso al que pertenece, en caso de un Profesor, incluye datos personales y la asignatura por la que es responsable.

En Curso.java, el profesor jefe se almacena como una referencia a un objeto Profesor, el constructor, el getter y el setter, utilizan el tipo Profesor.

En GestionCursos.java, al agregar un curso con los datos del profesor, se crea un objeto Profesor y este objeto se entrega al constructor de Curso, en caso de ser modificado el profesor, se crea un nuevo objeto y se actualiza la referencia mediante el setter setProfesorJefe().

Sobreescrituras

La sobreescritura de Alumno se utiliza al buscar un alumno, en MenuAlumnos.java se llama a alumno.obtenerResumen() y se imprime el resultado en consola, en ventana se utiliza el mismo método para mostrar sus datos mediante el cuadro de diálogo.

La sobreescritura de Profesor se utiliza en el método obtenerDetalleCurso() de GestionCursos.java, el cual obtiene al profesor jefe mediante getProfesorJefe() y de existir llama a profesor.obtenerResumen() para incluir sus datos en el detalle del curso.

Ambas sobreescrituras se ejecutan en funcionalidades del sistema disponibles desde consola y desde la ventana.

Evidencia de uso

- [src/main/java/Persona.java](#), aproximadamente líneas 38-40.
- [src/main/java/Alumno.java](#), aproximadamente líneas 21-32.
- [src/main/java/Profesor.java](#), aproximadamente líneas 20-31.
- [src/main/java/Curso.java](#), aproximadamente líneas 8-16.
- [src/main/java/MenuAlumnos.java](#), aproximadamente líneas 73-75.
- [src/main/java/GestionCursos.java](#), aproximadamente líneas 49-52.

Evidencia principal: [Persona.java](#), [Alumno.java](#) y [Profesor.java](#)

SIA-7 Menú de inserción y listado

Requisito solicitado: Crear un menú del sistema con inserción manual y listado para ambas colecciones, en opciones separadas.

Menú de consola

Nuestro menú cuenta con una sección de administración la cual permite acceder a la gestión de alumnos y cursos.

En MenuAlumnos.java se presentan las opciones de:

- Agregar un alumno
- Mostrar un alumno
- Mostrar todos los alumnos
- Registrar un alumno en un curso

En MenuCursos.java se presentan las opciones de:

- Agregar un curso
- Listar todos los cursos
- Buscar un curso

Ventanas

Las mismas funcionalidades mencionadas anteriormente también están en nuestras ventanas realizadas en netbeans en forma de Jforms con Swing.

Evidencia

- [src/main/java/MenuAlumnos.java](#), aproximadamente líneas 23-29 y 40-94.
- [src/main/java/MenuCursos.java](#), aproximadamente líneas 20-56.
- [src/main/java/VentanaAlumnos.java](#).
- [src/main/java/VentanaCursos.java](#)

Evidencia principal: [MenuAlumnos.java](#), [MenuCursos.java](#), [VentanaAlumnos.java](#) y [VentanaCursos.java](#).

SIA-8 Menú de edición eliminación y búsqueda

Requisito solicitado: Crear opciones separadas para editar, eliminar y buscar elementos de ambas colecciones.

Operaciones implementadas

1. **Buscar alumno:** busca un alumno mediante su RUT o nombre y apellido, con el método `obtenerAlumno`, retorna el objeto encontrado.
2. **Editar alumno:** solicita el RUT, busca el alumno y modifica su nombre y/o apellido mediante uso de setters heredados de `persona`, RUT no es modificable.
3. **Eliminar alumno:** busca un alumno mediante su RUT, si pertenece a un curso, lo elimina de la lista de alumnos del curso y después lo elimina de la lista de alumnos global.
4. **Buscar curso:** utiliza el código como key para el `HashMap` de cursos, si lo encuentra, muestra sus datos.
5. **Editar curso:** busca el curso por código, modifica su nombre y profesor jefe, el código no es modificable.
6. **Eliminar curso:** busca el curso por código, antes de eliminarlo, asigna null al curso de cada alumno de su lista de alumnos, luego lo elimina del `HashMap` de cursos.

Validaciones

Los métodos `obtenerAlumno` y `obtenerCurso` cuentan con excepciones propias, `AlumnoNoEncontradoException` y `CursoNoEncontradoException`, respectivamente, los menús capturan estas excepciones y muestran un mensaje correspondiente sin cerrar el programa.

En las ventanas se aplica validación de input con `trim`, los métodos de edición y eliminación retorna false en caso de no encontrar el elemento a eliminar.

Los inputs inválidos para las opciones de los menús de consola se controlan mediante `NumberFormatException`.

Evidencia

- [src/main/java/VentanaAlumnos.java](#) aproximadamente linea 276-317
- [src/main/java/VentanaCursos.java](#) aproximadamente linea 176-220
- [src/main/java/MenuCursos.java](#) aproximadamente linea 75-80
- [src/main/java/MenuAlumnos.java](#) aproximadamente linea 74-87
- [src/main/java/GestionCursos.java](#) aproximadamente linea 23-100
- [src/main/java/GestionAlumnos.java](#) aproximadamente linea 25-131

Evidencia principal: [GestionAlumnos.java](#), [GestionCursos.java](#)

SIA-9 Funcionalidad de utilidad para el negocio

Requisito solicitado: Implementar al menos una funcionalidad útil para el negocio, distinta de inserción, edición, eliminación o reportes, considerando un subconjunto filtrado por criterio.

Funcionalidad

Nuestro programa permite obtener alumnos que estuvieron ausentes en una fecha específica.

El usuario puede ingresar una fecha y nuestro programa recorre todos los registros. Luego aplica un filtro en el que conserva solo los que cumplen con que la fecha coincida con la ingresada y que el alumno no esté marcado como presente.

Luego de este filtrado, se muestra el RUT y el nombre de cada alumno ausente en esa fecha. Si ningún registro cumple muestra un mensaje diciendo que no hay alumnos ausentes en dicha fecha.

Utilidad

Esta funcionalidad es útil para que un colegio pueda identificar de manera rápida que alumnos faltaron durante un día específico y realizar el seguimiento correspondiente.

Evidencia

Tenemos el filtrado en `GestionRegistroAsistencia.java`, está en el método `obtenerAusentesPorFecha()`, líneas 239 y 264 aproximadamente.

La opción está tanto en ventana como en menú de consola.

[GestionRegistroAsistencia.java](#), método `obtenerAusentesPorFecha()`.

[MenuAsistencia.java](#), opción de alumnos ausentes por fecha.

[VentanaAsistencia.java](#), consulta mediante interfaz gráfica.

[SIA-9: Mostrar alumnos ausentes por fecha #40](#).

SIA-10 Consola y ventanas

Requisito solicitado: Implementar todas las funcionalidades mediante consola y ventanas, solicitando al inicio el modo de uso.

Selección inicial

El programa comienza en **GestionAsistencia.main()**, donde se crea una instancia de cada clase gestora, instancias que pasa a la clase **Menu**, este muestra tres opciones (usar consola, usar ventana, salir), si el usuario selecciona la consola se llama al método **mostrarMenu** y comienza la navegación por consola, si el usuario selecciona la ventana, se crea una instancia de **VentanaPrincipal**, se le entregan las clases gestoras y se muestra al usuario mediante **setVisible(true)**, finalmente el usuario puede terminar el menú y con ello, el programa.

Paridad funcional

La funcionalidad esperada está disponible en ambas interfaces, esto es comprobable al ver que las clases de menú y las clases de ventana correspondientes (**MenuAlumnos**, **VentanaAlumnos**, **MenuCursos**, **VentanaCursos**, **MenuAsistencia**, **VentanaAsistencia**, **MenuAvanzado**, **VentanaAvanzado**) utilizan y tienen acceso a los mismos métodos de las clases de gestión (**GestionCursos**, **GestionAlumnos**, **GestionAsistencia**, **GestionRegistroAsistencia**).

Evidencia

- [src/main/java/GestionAsistencia.java](#) aproximadamente línea 7-32
- [src/main/java/Menu.java](#) aproximadamente línea 27-61
- [src/main/java/MenuPrincipal.java](#) aproximadamente línea 10-19
- [src/main/java/VentanaPrincipal.java](#) aproximadamente línea 15-34
- [src/main/java/VentanaAdministracion.java](#) aproximadamente línea 18-48

Evidencia principal: [GestionAsistencia.java](#), [Menu.java](#), [MenuPrincipal.java](#), [VentanaPrincipal.java](#), [VentanaAdministracion.java](#)

SIA-11 Persistencia de datos

Requisito solicitado: Persistir datos mediante archivo de texto, CSV, Excel o DBMS local, cargando al inicio y guardando al salir mediante un sistema batch.

Formato y archivos

El sistema utiliza archivos de texto en formato **CSV**, en estándar UTF-8, estos archivos se encuentran en la carpeta **datos**, ubicada en la raíz del proyecto.

1. **datos/cursos.csv**: almacena el código, nombre y profesor jefe de cada curso.
2. **datos/alumnos.csv**: almacena el RUT, nombre, apellido y código del curso asignado a cada alumno.
3. **datos/asistencias.csv**: almacena el RUT del alumno, fecha, presencia, retiro anticipado, falta justificada y los motivos relacionados.

Se hace uso de **Path**, **Files**, **BufferedReader** y **BufferedWriter**, junto con un método **formatear** y **limpiar** para control de formato.

Carga inicial

El método `cargarTodo` carga primero los cursos, después los alumnos y finalmente las asistencias. Este orden permite asociar los alumnos a sus cursos y luego cargar los registros de asistencia. Si la carpeta **datos** no existe, el sistema la crea.

Guardado final

En **GestionAsistencia.main()** se registra un **shutdownHook**, este bloque se ejecuta cuando el programa finaliza de forma normal y llama al método **guardarTodo**, el cual crea la carpeta **datos** si no existe y llama tres otros tres métodos, **guardarAlumnos**, **guardarCursos** y **guardarAsistencias**, los cuales abren su archivo respectivo con **BufferedWriter** y se reescriben los archivos csv con los datos contenidos en las colecciones de la sesión actual, lo cual permite incluir los cambios que se hayan realizado a las colecciones.

Evidencia

- [src/main/java/CSV.java](#) aproximadamente linea 6-202
- [src/main/java/GestionAsistencia.java](#) aproximadamente linea 8-29
- [datos/cursos.csv](#)
- [datos/alumnos.csv](#)
- [datos/asistencias.csv](#)

Evidencia principal: [CSV.java](#), [GestionAsistencia.java](#)

SIA-12 Excepciones propias y try catch

Requisito solicitado: Diseñar y codificar dos excepciones que se utilicen en el programa. Manejar las excepciones mediante try catch.

Excepciones

1. **AlumnoNoEncontradoException:** se utiliza cuando una búsqueda de alumno por RUT no encuentra al alumno determinado, es subclase de **Exception**.
2. **CursoNoEncontradoException:** se utiliza cuando una búsqueda por código no encuentra el curso determinado, es subclase de **Exception**.

Manejo

En consola, los menús son los responsables de capturar las excepciones, estos colocan la lectura y ejecución de las opciones dentro de bloques **try-catch**, el mensaje se imprime por consola.

En ventana, las ventanas **Swing** son las responsables de capturar las excepciones dentro de los métodos asociados a los botones, el mensaje es mostrado en un cuadro de diálogo.

Recuperación

En consola, los bloques **try-catch** de los menus estan dentro de ciclos **do-while**, por lo que cuando ocurre un error recuperable, el bloque **catch** muestra el mensaje por consola y la ejecución llega al final de ciclo, el menú vuelve a comenzar y se solicita una opción nuevamente, evitando cerrar el programa.

En ventana, la excepción solo termina la acción del botón que se está ejecutando, la ventana permanece abierta dándole la oportunidad al usuario de corregir el dato o cambiar de operación.

Evidencia

- [src/main/java/AlumnoNoEncontradoException.java](#) aproximadamente linea 2-7
- [src/main/java/CursoNoEncontradoException.java](#) aproximadamente linea 2-7
- Los menús correspondientes, están envueltos en bloque try-catch.

Evidencia principal: [CursoNoEncontradoException.java](#), [AlumnoNoEncontradoException.java](#)

SIA-13 Uso de GitHub

Requisito solicitado: Utilizar GitHub realizando al menos seis commits.

Historial

A fecha de 2 de Octubre de 2026, el repositorio tiene más de 100 commits. Estos incluyen cambios de documentación, correcciones e implementación de funcionalidades del proyecto.

Organización

El trabajo fue dividido en **issues** semanales acorde al avance SIA recomendado en la planificación del curso.

El repositorio mantiene cuatro ramas, la rama main la cual contiene la versión integrada y compartida del proyecto, la rama **nicolasj** la cual es la rama personal utilizada por Nicolas Jaramillo, la rama **lucasahu** la cual es la rama personal utilizada por Lucas Ahumada y la rama **alessandror** la cual es la rama personal de Alessandro Reginato.

Cada integrante desarrolló sus **issues** en su propia rama, una vez completado, se integraba a la rama main por medio de **pull request** y **merge**, este flujo permite trabajar de forma paralela, evitando introducir cambios disruptivos para el código de la rama main.

Evidencia

Evidencia principal: [Repositorio de GitHub](#), [Issues](#), [Ramas](#), [Historial de commits](#).